

Lab Manual — Notes for Teaching Assistants

DSA27 · Weeks 1 to 3 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026

Contents

1. The three labs in one page	1
2. Running a session (2 hours)	2
3. What to demonstrate	2
4. Exercise notes	3
5. Checking students' work	7
6. Environment problems you will see	7
7. Answers to the take-home practice	8

*This guide is for the TAs running the Week 1–3 labs. Students can read it too — there is nothing here they should not see. It does **not** contain solutions: it contains what to demonstrate, where students go wrong, and how to check their work. Solutions are only useful to someone who has already tried.*

1. The three labs in one page

Week	Lab	Core idea to land	The trap to demonstrate live
1	Interpreter, numbers, strings, lists	A name refers to an object	<code>b = a;</code> <code>b.append(4)</code> changes <code>a</code>
2	Control flow, functions, errors, modules	Functions return; errors are information	the mutable default <code>def f(a, L=[])</code>
3	Data structures, classes, generators	Each structure has a cost; a class bundles data with its contract	the shared class variable <code>tricks = []</code>

Each lab has a reading (docs/labs/labNN-*.md, also as a PDF in docs/pdf/), a starter file (labs/labNN.py) and a test file (tests/test_labNN.py).

Where students are coming from. AI students have taken CS1002 Object Oriented Programming. Bio and SWE students have taken CS012, structured programming only, often in C++. Expect the Bio and SWE groups to struggle more in Lab 03 (classes) and the AI group to be overconfident in Lab 01 (they will get the floor-division and aliasing checkpoints wrong anyway). Pair a stronger student with a weaker one for the Checkpoints if the room allows it.

2. Running a session (2 hours)

Time	What	Notes
0:00–0:10	Attendance; environment problems	Attendance counts toward the 75% rule — record it every session.
0:10–0:35	Live demonstration at the REPL	Follow section 3 below. Project a terminal, large font. Type; do not paste.
0:35–1:40	Students work: Checkpoints, then exercises	Walk the room. Ask “what did you predict?” before “what did you get?”
1:40–2:00	Check-off	Each student shows <code>pytest tests/test_labNN.py</code> and explains one line you choose.

Principles for helping.

- Ask them to **read the failing test’s output aloud** before you say anything. Half the time they then see it.
- Point to the **docstring** and the **test file** before you give a hint. The habit of reading the specification is the lesson.
- Give the smallest hint that unblocks — the hints in each lab’s “Exercises” part are graded from small to large. Do not type on a student’s keyboard.
- When a student says “it works on my machine”, run the tests. The tests define “works”.

3. What to demonstrate

Lab 01 — Week 1

1. Start the REPL; `2 + 2`; `_`; `type(5)`, `type(5.0)`; `8 / 4` is `2.0`.
2. `-7 // 2` and `-7 % 2`. Ask the room first; most will say `-3` and `-1`.
3. `2 ** 100`, then `0.1 + 0.2 == 0.3`. One sentence on why: binary fractions.
4. `word = 'Python'`; index, negative index, slice; draw the “edges between characters” diagram on the board; `word[: :-1]`.

5. **The main event.** `a = [1, 2, 3]; b = a; b.append(4); a`. Then `a is b`, `id(a)`, `c = a[:]`, `c is a`, `c == a`. Draw the arrows on the board.
6. `[[0] * 3] * 3` and set one cell. Leave it unresolved — Lab 03 fixes it.
7. Open `labs/lab01.py`, write `celsius_to_fahrenheit` **wrongly** (no round), run `pytest tests/test_lab01.py -k celsius`, read the failure together, fix it. This is the only exercise you solve in front of them.

Lab 02 — Week 2

1. `if/elif` order: the score `>= 50` before `score >= 90` example.
2. `0` or `'default'`; `[]` and `1/0` — short-circuiting.
3. `for` over a string; `range(5)` is not a list; `enumerate`.
4. The prime-search with `for ... else`. Ask: which statement does the `else` belong to?
5. **The main event.** `def f(a, L=[])` called three times. Then the `None` fix. Then `add_item` versus `replace` — mutation versus rebinding.
6. Run `trace_demo.py` (Part 6.2) and read the traceback bottom-up, aloud.
7. `count_single / count_nested` for `n = 10, 100, 1000` — tie it to Lecture 02.

Lab 03 — Week 3

1. `marks.sort()` returns `None`; `sorted` does not. Everyone has hit this.
2. A list as a stack; then `pop(0)` versus `deque.popleft()`. Optional: time 100,000 `pop(0)` against 100,000 `popleft()`.
3. A comprehension next to the loop it replaces. Then the correct grid: `[[0] * 3 for _ in range(3)]` — closing the loop from Lab 01.
4. `{}` is a dict; `set()` is a set. `x in list` versus `x in set` on a million items with `time.perf_counter`.
5. The counting pattern with `get`, then `setdefault`.
6. **The main event.** Build `Dog` live; then `BadDog` with `tricks = []`. Then `Fraction` with only `__init__` and show `Fraction(1, 2)` prints as `<__main__.Fraction object at 0x...>`; add `__repr__` and show the difference; add `__eq__` and show `==` start working.
7. A three-line generator and `next()` by hand until `StopIteration`.

4. Exercise notes

For each exercise: what it tests, and the mistakes you will see most. The tests are the definition of correct — when in doubt, read `tests/test_labNN.py`.

Lab 01

Exercise	What it is really testing	Common mistakes
seconds_to_hms	// and % twice; :02d	seconds / 3600 (a float); padding the hours; forgetting that minutes are seconds % 3600 // 60, not seconds // 60
celsius_to_fahrenheit	float arithmetic, round(x, 1)	printing instead of returning; 9 // 5 (integer division gives 1)
split_evenly	returning two values	returning a list [a, b] — the test compares with a tuple, and [33, 1] != (33, 1)
initials	split() with no argument; building a string in a loop	split(' '), which produces empty strings for repeated spaces and then ''[0] raises IndexError
is_palindrome_word	strip, lower,[::-1]	returning 'True' (a string) or 1; the test uses is True
mask_email	index('@'), slicing, '*' * n	off-by-one in the number of stars; forgetting the short-name case
middle	slice arithmetic	a chain of ifs that breaks for length 0 or 1; the one-slice answer exists
rotate_left	%, two slices, not mutating	k % 0 on the empty list; mutating the input with pop/append — a test checks the original is unchanged
fib_list	the tutorial loop, collecting into a list	an off-by-one: n = 1 must give [0]
collatz_steps	a while with an if inside	counting the starting number as a step; n / 2 (a float — it still terminates, but the habit is wrong)

Rule for Lab 01: no for, no import. A student who uses for from prior knowledge has not failed — but ask them to rewrite one function with while, because the point of the lab is the while-loop mechanics they will need for binary search in Week 8.

Lab 02

Exercise	What it is really testing	Common mistakes
<code>classify_triangle</code>	validation before classification; raise <code>ValueError</code>	<code>>=</code> versus <code>></code> in the inequality (1, 2, 3 is degenerate and must be rejected); checking only one of the three inequalities; returning an error string
<code>course_result</code>	order of conditions	checking the total before attendance; <code>final < 30</code> (it is 30% of 60 , i.e. 18); not validating ranges
<code>fizzbuzz</code>	<code>for, %</code>	testing <code>% 3</code> before <code>% 15</code> , so 15 becomes "Fizz"; returning ints instead of strings
<code>is_prime</code>	early return; the $\sqrt{n}$ bound	<code>range(2, int(n ** 0.5))</code> — misses the square root itself, so 9 and 25 come out prime; forgetting <code>n < 2</code>
<code>primes_below</code>	reuse	re-implementing the primality test inline
<code>first_repeated</code>	nested loop, early exit	returning the first value that <i>has</i> a duplicate somewhere later, instead of the value whose <i>repeat comes first</i> : for [2, 5, 5, 2] that gives 2, but the answer is 5. Ask them to trace it by hand
<code>count_pairs</code>	counting, then a closed form	the formula <code>n * (n - 1) / 2</code> returns a float ; they need <code>//</code> . Both loop and formula pass — ask which is $O(1)$
<code>calculator</code>	<code>match</code> , capture patterns, <code>*rest</code>	case <code>["sum", *rest]</code> accepts "sum" with no numbers — the test requires <code>ValueError</code> ; a bare <code>except:</code> that also swallows their own bugs
<code>parse_int</code>	<code>try/except ValueError</code>	<code>text.isdigit()</code> checks (fails on " -7 "); catching <code>Exception</code>
<code>stats</code>	<code>*args</code>	starting min/max at 0; using min/max (not allowed); integer division for the mean

Exercise	What it is really testing	Common mistakes
<code>append_to</code>	the mutable default	writing <code>target=[]</code> — the second test call fails, which is exactly the lesson
<code>make_multiplier</code>	returning a function	returning <code>k * x</code> with <code>x</code> undefined; calling the lambda instead of returning it
<code>apply_n</code>	passing a function	calling <code>f</code> once; recursion without a base case for <code>n = 0</code>

Lab 03

Exercise	What it is really testing	Common mistakes
<code>unique_in_order</code>	set for membership, list for order	<code>if v not in result</code> — $O(n^2)$; the 400,000-item test takes minutes. Let them see it hang, then ask why. <code>list(set(values))</code> loses the order
<code>word_frequencies</code>	<code>split</code> , <code>strip(chars)</code> , <code>get</code>	<code>replace</code> to delete punctuation (also removes it from inside words); not skipping words that become empty
<code>top_k</code>	sorting by a tuple key	sorting by count only (ties come out in dictionary order); <code>reverse=True</code> on a tuple key reverses the alphabetical tie-break too
<code>transpose</code>	nested comprehension	the order of the two fors; the empty matrix (<code>matrix[0]</code> raises)
<code>invert</code>	grouping	overwriting instead of appending (<code>out[v] = k</code>); forgetting to sort the key lists
<code>common_elements</code>	<code>&</code>	returning a set instead of a sorted list
<code>group_by_length</code>	<code>setdefault</code> or <code>get</code>	sorting the words (the test expects input order)
<code>Bag</code>	a class with state; dunders	see below
<code>countdown</code>	<code>yield</code>	building and returning a list — the test checks it is a generator

Exercise	What it is really testing	Common mistakes
chunks	a generator with <code>range(0, n, size)</code>	no explicit size check: <code>range(0, n, 0)</code> happens to raise <code>ValueError</code> for size 0, but a negative size silently yields nothing — the test checks both; returning a list

The Bag class is the most important exercise of the three labs, because it is the shape of every `dsa/` class. Check, in this order:

1. The counts live in a **dictionary created in `__init__`**, not a class variable. (The test that two bags do not share state catches the class-variable version.)
2. `__len__` is $O(1)$: a running total, not `sum(self._counts.values())`. The tests cannot detect the $O(k)$ version — so ask the student what `len` costs. This is a “know-why” question, the kind the exam asks.
3. `remove` deletes the key when a count reaches zero, and raises `KeyError` — not `ValueError`, not `print`.
4. `__eq__` compares the dictionaries, and returns `NotImplemented` for non-bags.
5. `__repr__` is `Bag([...])` with items **sorted**, repeats included.

5. Checking students’ work

From the student’s DSA27 folder, with their virtual environment active:

```
git status                # nothing unexpected changed?
git diff --stat tests/    # tests must be unchanged – this must be empty
pytest tests/test_lab01.py -q  # the result you record
```

If tests were edited, restore them with `git checkout -- tests/` and run again. Record the result of the original tests.

Suggested marking, per lab (the lab portion of coursework is announced by the lecturer):

Mark	Meaning
2	All tests pass, and the student explains the line you point at
1	Most tests pass, or all pass but the explanation is weak
0	Little working, or the student cannot explain their own code

The explanation matters more than the count. A student who passes 8 of 10 and can explain every line has learnt more than one who passes 10 of 10 with code they cannot read.

6. Environment problems you will see

Symptom	Cause	Fix
python opens the Microsoft Store	Windows App Execution Aliases	<i>Settings</i> → <i>Apps</i> → <i>Advanced app settings</i> → <i>App execution aliases</i> : switch off python.exe and python3.exe
pytest is not recognised	venv not activated	<code>.\.venv\Scripts\activate</code> ; the prompt must start with <code>(.venv)</code>
activate refused: "running scripts is disabled"	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , then activate again
ModuleNotFoundError: No module named 'labs' VS Code runs a different Python	running from the wrong folder wrong interpreter selected	cd to the DSA27 folder — the one with <code>pytest.ini</code> in it <code>Ctrl+Shift+P</code> → <i>Python: Select Interpreter</i> , then <code>.\.venv\Scripts\python.exe</code> as above; <code>python -c "import sys; print(sys.prefix)"</code> in both
Tests pass in the terminal but not in VS Code (or the reverse)	two interpreters	<code>exit()</code> and restart the REPL, or just use <code>pytest</code>
Changes to <code>labs/lab01.py</code> have no effect in the REPL A test "hangs"	the module was already imported an infinite while loop, or $O(n^2)$ in <code>unique_in_order</code>	<code>Ctrl+C</code> ; the traceback shows the line it was on
graphviz tests fail in <code>pytest -m "not challenge"</code>	the dot binary is not installed or not on PATH	install Graphviz from https://graphviz.org/download/ and reopen the terminal. Not needed for the labs themselves
<code>git clone</code> asks for a password	HTTPS cloning of a public repo should not — they probably mistyped the URL	copy the URL from the README

7. Answers to the take-home practice

The take-home items are ungraded and discussed at the start of the following lab. The quick answers:

- **Lab 01, item 3:** `x` is `[1, [2, 3, 4]]` — `y[0] = 9` changed only the copy; the inner list is shared.
- **Lab 01, item 5:** the time roughly doubles when the input doubles — linear.
- **Lab 02, item 3:** 20 iterations for 1,000,000 — one per binary digit, $\text{floor}(\log_2 n) + 1$.

- **Lab 02, item 4:** 7 guesses — $\text{ceil}(\log_2 101)$ — the same halving as Lab 01's Checkpoint 4 and, in Week 8, binary search.
- **Lab 02, item 5:** yes, a straight line for primes: $O(n)$.
- **Lab 03, item 1:** sorting is $O(n \log n)$; counting (dictionary or Bag) is $O(n)$.
- **Lab 03, item 3:** zip stops at the shorter iterable, and a generator only computes a value when asked, so only ten are ever produced.
- **Lab 03, item 5:** slicing copies the rest of the list at every call, so the sliced version is $O(n^2)$ in total and the index version $O(n)$.